Text-to-LoRA: Background, Class-Scale Recreation, and Extensions

Victoria Lee
Deep Learning Midterm Project
`Text-to-LoRA recreation (arXiv:2506.06105)`

Abstract

Foundation models are typically specialized with parameter-efficient fine-tuning such as Low-Rank Adaptation (LoRA), which still requires per-task datasets, training runs, and hyperparameter search. *Text-to-LoRA* (T2L) Charakorn et al. [2025] instead trains a hypernetwork that emits task-specific LoRA weights from a natural-language task description in a single forward pass. This midterm report summarizes the original paper, describes our independent recreation on an Apple M5 Pro with 24 GB unified memory, and outlines extensions for the final project. We implement both reconstruction distillation of oracle LoRAs and end-to-end supervised fine-tuning (SFT) of the hypernetwork, with L/M/S architectural variants, Bias-HyperInit, z-scored reconstruction targets, and text conditioning via a frozen sentence encoder.

1 Introduction

Large language models (LLMs) exhibit broad capabilities but specialization poses challenges due to expensive compute and data requirements. Freezing base weights and learning a small adapter, as introduced in LoRA Hu et al. [2022], is the most common approach, but is still costly in data, compute, and engineering overhead.

Charakorn et al. Charakorn et al. [2025] ask whether many LoRA adapters share an underlying adaptation mechanism that a neural network can compress and, at test time, decode from natural language alone. Their answer is **Text-to-LoRA (T2L)**: a hypernetwork Ha et al. [2016] conditioned on a text embedding of the task description that generates LoRA matrices A and B for selected query and value projections across all layers in one batched forward pass, substantially reducing per-task adaptation cost at inference time.

Midterm goals. For this course project we (i) review the paper's motivation, method, and empirical claims; (ii) reimplement the core method at class scale with newly written code; and (iii) propose extensions for the final project.

2 Background: The Original Paper

2.1 Problem setting

Given tasks $\mathcal{T} = \{t^1, \ldots, t^T\}$ with fine-tuning sets $\mathcal{D}^i$ and natural-language descriptions Z^i, standard LoRA solves

$$\Delta W^i = \arg\min_{\Delta W^i} \mathcal{L}_{\text{SFT}}(\mathcal{D}^i, \Psi, \Delta W^i), \tag{1}$$

where Ψ denotes frozen pretrained weights and $\Delta W = BA$ with low rank r. T2L instead learns hypernetwork parameters θ such that

$$\Delta W^i_{m,l} = h_\theta(\phi^i_{m,l}), \qquad \phi^i_{m,l} = \text{concat}\big[f(z^i),\, E[m],\, E[l]\big], \tag{2}$$

Preprint.

where $f(z)$ is a text embedding (e.g., CLS from `gte-large-en-v1.5`), and $E[m]$, $E[l]$ are learned embeddings for module type and layer index.

2.2 Architectures (L / M / S)

Hypernetwork capacity is mostly determined by the output head. The paper proposes three inductive biases (Fig. 2 / App. F in Charakorn et al. [2025]):

- **L**: head emits both A and B ($2 \cdot r \cdot d$ outputs).
- **M**: shared head emits either A or B, selected by an A/B embedding ($r \cdot d$).
- **S**: head emits one rank vector at a time, with A/B and rank embeddings (d).

All variants share a backbone in which the task encoder $\rightarrow$ mixer MLP $\rightarrow$ residual MLP blocks $\rightarrow$ final MLP, with Bias-HyperInit Beck et al. [2023] for stable training (A-like uniform bias, B zeros; scaled for M/S).

2.3 Training: Reconstruction vs. SFT

For reconstruction, given a library of pretrained LoRAs Ω, we minimize elementwise L1 between $h_\theta(\phi)$ and target adapters. The paper predicts *z-scored* weight entries and denormalizes at inference. Reconstruction recovers oracle benchmark LoRAs (Table 1) but struggles at zero-shot generalization when similar tasks have distant adapters in weight space (Table 6, App. D).

For SFT, we optimize θ end-to-end with language-modeling loss through the frozen LLM plus generated LoRA (Eq. 5), sampling a description per example. SFT on hundreds of Super-NaturalInstructions (SNI) Wang et al. [2022] tasks yields zero-shot adapters that beat multi-task LoRA and approach task-specific oracles on several benches (Table 2).

2.4 Key empirical takeaways

1. Lossy compression of many LoRAs is possible while retaining much of oracle accuracy.
2. Zero-shot generation requires text conditioning and benefits strongly from SFT rather than just reconstruction.
3. Performance scales with number of training tasks when compute is also scaled accordingly.
4. Generated adapters cluster by task semantics in activation space (t-SNE); thus, description alignment matters (Table 5).

3 Recreation

3.1 Design constraints and fidelity

We use an Apple M5 Pro with 24 GB unified memory, so compute is greatly downscaled relative to the paper, but the general approach is preserved: hypernetwork $\rightarrow$ LoRA A/B from text, L/M/S heads, Bias-HyperInit, reconstruction via z-scored weights, and SFT through injected LoRA. Our experiments use the M variant, a frozen all-MiniLM-L6-v2 description encoder, and rank-8 adapters targeting `q_proj` and `v_proj`.

The code is our independent implementation (package `t2l/`, single CLI `run.py`) of the method described in the paper; it does not copy the public repository.

Table 1: Paper setup vs. our class-scale recreation.

	Paper	This project
Base LLM	Mistral-7B-Instruct	Qwen2.5-1.5B-Instruct
Embedder	gte-large-en-v1.5	all-MiniLM-L6-v2
Default variant	M (reported)	M
Oracle tasks	9 benches	4 (BoolQ, ARC-E, PIQA, OpenBookQA)
SFT corpus	479 SNI tasks	16 classifier slots in reported run
SFT steps	up to $\sim$1M	2,500
Hardware	H100 80 GB	Apple M5 Pro, 24 GB unified
Descriptions	GPT-4o mini	templates + paraphrases

3.2 Implementation overview

task text $\xrightarrow{f}$ CLS $\oplus$ layer/module emb (+ A/B, rank for M/S)
$\rightarrow$ residual MLP backbone $\rightarrow$ head $\rightarrow$ LoRA A, B
$\rightarrow$ inject into frozen LLM $\rightarrow$ L1 (recon) or CE (SFT)

Figure 1: Method sketch matching Charakorn et al. [2025], implemented in `t2l/model.py`.

Hypernetwork (`t2l/model.py`). `HyperModulator` implements the L/M/S output designs; the reported experiments use M. `TaskEmbedder` wraps a frozen all-MiniLM-L6-v2 encoder Wang et al. [2020], Reimers and Gurevych [2019] and mean-pools its token representations into a 384-dimensional description embedding. `DynamicLoRALinear` replaces target linears so generated A, B remain on the autograd graph during SFT (buffers would block gradients).

Pipeline (`run.py`).

1. `descriptions` — build description banks.
2. `oracles` — train per-task PEFT LoRAs; pack A/B tensors.
3. `recon` — distill z-scored oracles with text (or one-hot) conditioning; early-stop on L1.
4. `sft` — end-to-end cross-entropy optimization through the frozen base model and generated adapters.
5. `multitask` / `eval` — baselines, zero-shot T2L, description ablation, t-SNE of task-encoder activations.

Memory recipe. The frozen Qwen2.5-1.5B-Instruct base model runs in FP16 through Apple's MPS backend. SFT uses batch size 1 and no gradient checkpointing; reconstruction does not back-propagate through the language model and is comparatively cheap.

3.3 Experimental protocol (midterm)

We evaluate on BoolQ, ARC-Easy, PIQA, and OpenBookQA using 500 examples per benchmark and report:

- **Reconstruction recovery**: recon-T2L accuracy vs. task-specific oracle LoRAs (Table 1 analog).
- **Zero-shot SFT-T2L** vs. the frozen base model (a reduced Table 2 analog).
- **Description ablation**: aligned vs. random / cross-task text (Table 5 analog).
- **Qualitative analysis**: t-SNE clustering of encoder activations by task.

Training setup. Each oracle adapter was trained for 300 optimizer steps with learning rate 2×10^{-4}, batch size 1, sequence length 256, and at most 1,000 examples. Reconstruction used a maximum of 5,000 steps, learning rate 10^{-3}, batch size 2, and eight descriptions per task. SFT used 2,500 optimizer steps, learning rate 5×10^{-5}, batch size 1, sequence length 192, 128 examples per task, and 16 descriptions per task. LoRA rank was 8 with $\alpha = 16$ and dropout 0.05.

Absolute numbers are not expected to match the 7B / 479-task paper; we instead test whether reconstruction recovers oracle performance and whether generated representations depend on task descriptions.

4 Results

4.1 Benchmark accuracy

Table 2: Accuracy (%) on 500 examples per benchmark. The reconstructed T2L adapters recover the task-specific oracle performance, whereas the initial SFT run fails to transfer to the QA output formats.

Method	BoolQ	ARC-E	PIQA	OpenBookQA	Avg.
Base model	73.8	79.6	63.4	68.4	71.3
Oracle LoRA	69.4	90.2	74.0	77.4	77.8
Reconstructed T2L	69.0	90.2	74.0	**78.0**	**77.8**
SFT T2L (initial)	0.0	0.0	5.2	0.0	1.3

Reconstruction is the strongest replication result. Its 77.8% average essentially matches the oracle average and improves on the base model by 6.5 percentage points. The small gain over the oracle on OpenBookQA is consistent with the paper's observation that lossy reconstruction can act as regularization. BoolQ is an exception: both adapted models underperform the already strong base model.

The initial SFT result is a diagnosed failure mode rather than evidence of numerical instability. The generated adapter weights were finite, but the meta-training corpus contained raw numeric classification labels. At evaluation time, the adapted model therefore emitted repeated numeric strings such as `0000` or `0100`, which cannot match Boolean answers or letter-valued multiple-choice answers. This explains the 5.2% PIQA score and zero exact-match scores elsewhere. We subsequently revised the SFT pipeline to use 16 unique, format-diverse tasks, meaningful class labels, reserved target-token space, and explicit end-of-sequence supervision; results from retraining are not included in Table 2.

Description alignment. On BoolQ, reconstruction T2L obtained 74% with an aligned description, compared with 67% for a random string and 61% for a description from another task. Although this is a single-task ablation, the ordering supports the paper's claim that generated adapters depend on description–task alignment.

4.2 Task-encoder representation

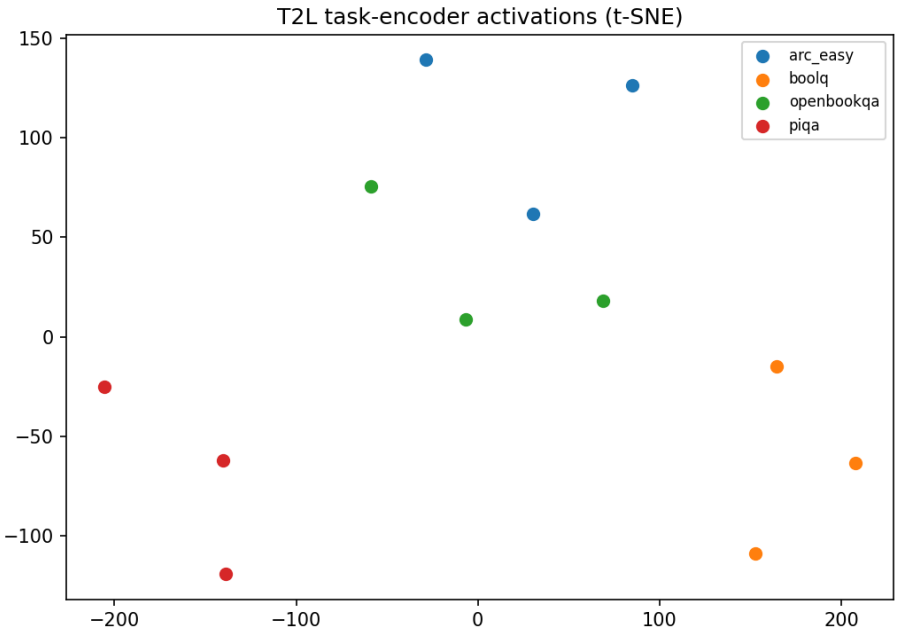


Figure 2: t-SNE projection of task-encoder activations for three descriptions of each evaluation task. Same-task descriptions occupy distinct regions, showing that the conditioning pathway retains task-specific information. The axes have no intrinsic interpretation, and separation alone does not establish adapter quality.

Figure 2 shows clear task-dependent structure: BoolQ and PIQA occupy separated lower regions, while ARC-Easy and OpenBookQA lie primarily in the upper portion of the projection. The three descriptions for each task remain distinguishable from most other tasks. This provides qualitative evidence that the text-conditioning pathway encodes task identity, but the failed initial SFT accuracy demonstrates that representation separation is not sufficient for useful adapter generation.

5 Limitations of the recreation

- Smaller base model and far fewer SFT tasks/steps reduce zero-shot headroom.
- The reported SFT checkpoint used eight unique classification datasets repeated across 16 task slots. This narrow distribution and its numeric label format caused output-format collapse. The revised pipeline uses 12 unique training tasks and four held-out tasks spanning classification, entailment, Boolean QA, and multiple-choice QA, but has not yet been evaluated.
- Template paraphrases are weaker than GPT-4o-mini description banks.
- Full Arrow Routing, Hyperdecoder, and evaluated multi-task LoRA baselines are omitted.
- t-SNE preserves local neighborhoods but can distort global geometry; its axes and absolute distances should not be interpreted directly.

6 Possible Extensions (Final Project)

We prioritize extensions that are feasible on 16–24 GB GPUs yet scientifically meaningful relative to the paper.

6.1 Extension A: Scale task diversity under fixed compute

Reproduce the paper's finding that more tasks help *only* when steps scale (Table 3 / App. C): sweep $\{4, 8, 12\}$ real Hugging Face tasks (or larger SNI subsets) at fixed vs. proportional step budgets; plot zero-shot benchmark average. **Deliverable:** scaling curves + short analysis of under-training vs. diversity.

6.2 Extension B: Reconstruction in ΔW space

App. K notes that $A \| B$ permutations break weight-space similarity, while flattened $\Delta W = BA$ correlates better with task embedding similarity. Train a recon head on BA (or SVD-aligned A, B) and compare zero-shot SFT-initialized-from-recon vs. scratch SFT. **Hypothesis:** ΔW-space recon improves transfer of the reconstruction library into the SFT regime.

6.3 Extension C: Steerable / compositional descriptions

Figure 4 in the paper shows answer-style steerability from wording alone. Build a controlled eval: same question, descriptions that request (i) short answer, (ii) chain-of-thought, (iii) wrong domain. Measure accuracy *and* format compliance. Optionally add a lightweight description rewriter (small local LM) before $f(z)$.

6.4 Extension D: Cross-model transfer

Paper limitation: whether T2L trained on a small sibling transfers within a family. Train hypernet on Qwen2.5-0.5B or 1.5B, evaluate generated LoRAs on 3B (same module layout) with frozen hypernet or a linear head remapping. **Deliverable:** transfer matrix across sizes.

6.5 Extension E: Beyond q/v LoRA

Replace or augment targets with MLP adapters / DoRA / rank patterns, or generate *activation* modulators instead of weights (discussion in Charakorn et al. [2025]). Even a single alternative output space (e.g., gate vectors per layer) would differentiate the final project from a pure replication.

6.6 Recommended final-project path

Primary: Extensions A + C (scaling + steerability) on real HF/SNI data with full eval tables and t-SNE.
Stretch: B or D if compute allows.
This yields a clear narrative: *faithful midterm recreation → final project probes scaling, language control, and/or transfer beyond the original experimental budget*.

7 Conclusion

Text-to-LoRA reframes adaptation as amortized inference over a hypernetwork conditioned on language. Our midterm recreation implements the core architectures and both training regimes with independent code at class scale on Qwen2.5-1.5B, running on an Apple M5 Pro with 24 GB unified memory. The final project will move from replication toward analysis—especially task scaling, description steerability, and optionally ΔW-space reconstruction or cross-size transfer.

Acknowledgments and Disclosure of Funding

Course midterm project report for Deep Learning. Method based on Charakorn et al., "Text-to-LoRA: Instant Transformer Adaption," 2025.

References

M. Beck, et al. A Survey of Meta-Reinforcement Learning. *arXiv:2301.08028*, 2023. Bias-HyperInit as used by Charakorn et al. Charakorn et al. [2025].

R. Charakorn, E. Cetin, Y. Tang, and R. T. Lange. Text-to-LoRA: Instant Transformer Adaption. *arXiv:2506.06105*, 2025. https://arxiv.org/abs/2506.06105.

D. Ha, A. Dai, and Q. V. Le. HyperNetworks. *ICLR*, 2017. https://arxiv.org/abs/1609.09106.

E. J. Hu, et al. LoRA: Low-Rank Adaptation of Large Language Models. *ICLR*, 2022.

N. Reimers and I. Gurevych. Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks. *EMNLP-IJCNLP*, 2019.

W. Wang, F. Wei, L. Dong, H. Bao, N. Yang, and M. Zhou. MiniLM: Deep Self-Attention Distillation for Task-Agnostic Compression of Pre-Trained Transformers. *NeurIPS*, 2020.

Y. Wang, et al. Super-NaturalInstructions: Generalization via Declarative Instructions on 1600+ NLP Tasks. *EMNLP*, 2022.